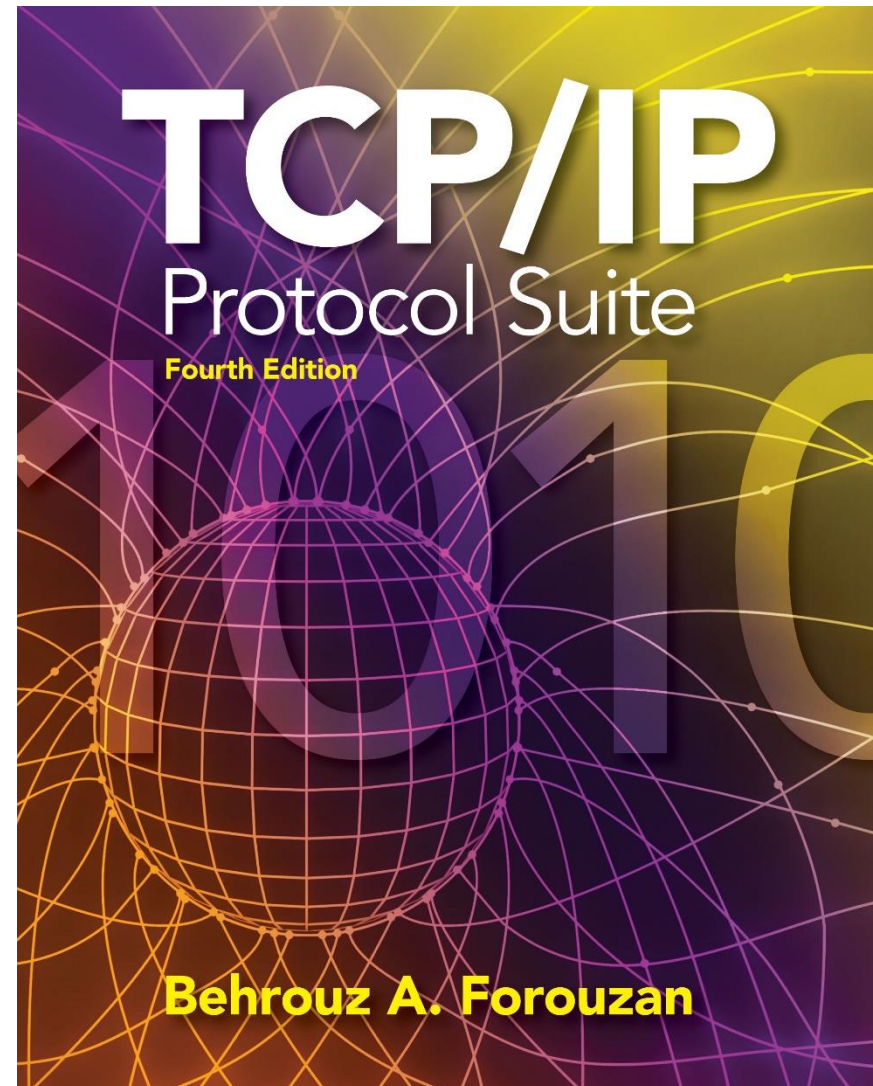


Chapter 8

Address Resolution Protocol (ARP)



OBJECTIVES:

- ☐ To make a distinction between logical address (IP address) and physical address (MAC address).
- ☐ To describe how the mapping of a logical address to a physical address can be static or dynamic.
- ☐ To show how the address resolution protocol (ARP) is used to dynamically map a logical address to a physical address.
- ☐ To show that the proxy ARP can be used to create a subnetting effect.
- ☐ To discuss ATMARF, which maps the IP addresses when the underlying network is an ATM WAN.
- ☐ To show that an ARP software package can be made of five components.
- ☐ To show the pseudocode for each module used in the ARP software package.

Chapter Outline

8.1 Address Mapping

8.2 The ARP Protocol

8.3 ATM ARP

8.4 ARP Package

8-1 ADDRESS MAPPING

The delivery of a packet to a host or a router requires two levels of addressing: *logical* and *physical*. We need to be able to map a logical address to its corresponding physical address and vice versa. These can be done using either *static* or *dynamic* mapping.

Topics Discussed in the Section

- ✓ **Static Mapping**
- ✓ **Dynamic Mapping**

8-2 ADDRESS MAPPING

Anytime a host or a router has an IP datagram to send to another host or router, it has the logical (IP) address of the receiver. But the IP datagram must be encapsulated in a frame to be able to pass through the physical network. This means that the sender needs the physical address of the receiver. A mapping corresponds a logical address to a physical address. ARP accepts a logical address from the IP protocol, maps the address to the corresponding physical address and pass it to the data link layer.

Topics Discussed in the Section

- ✓ **Packet Format**
- ✓ **Encapsulation**
- ✓ **Operation**
- ✓ **Proxy ARP**

Figure 8.1 *Position of ARP in TCP/IP protocol suite*

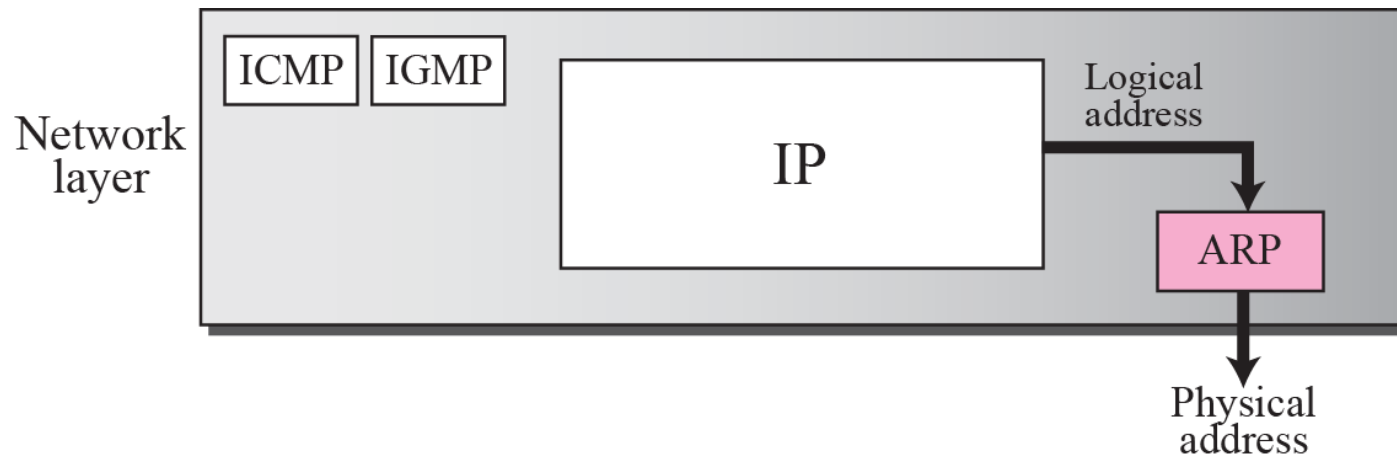
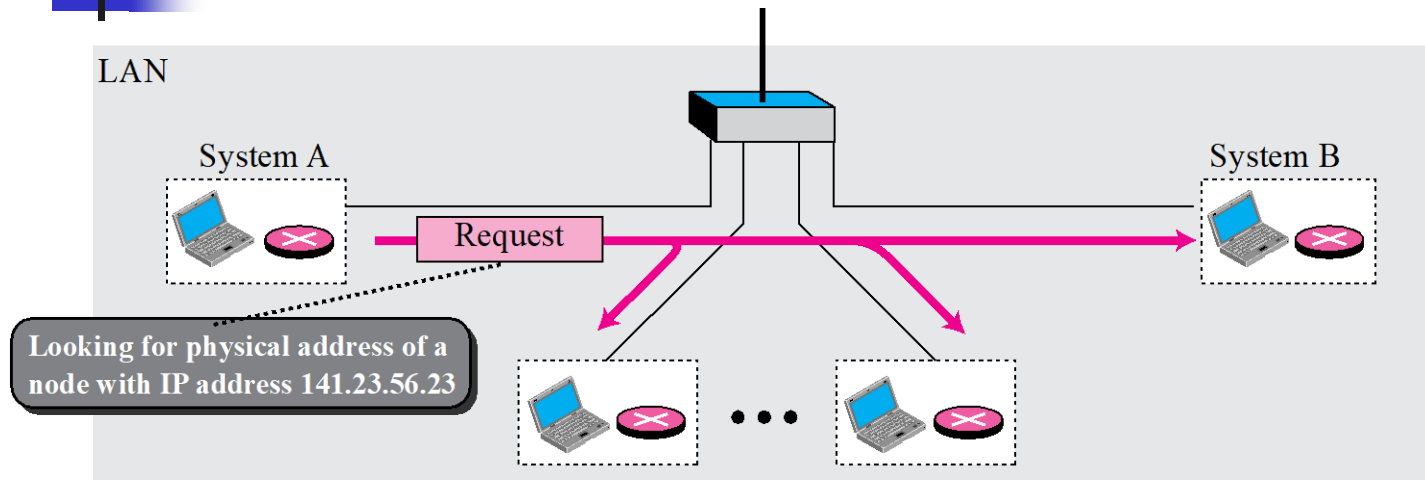
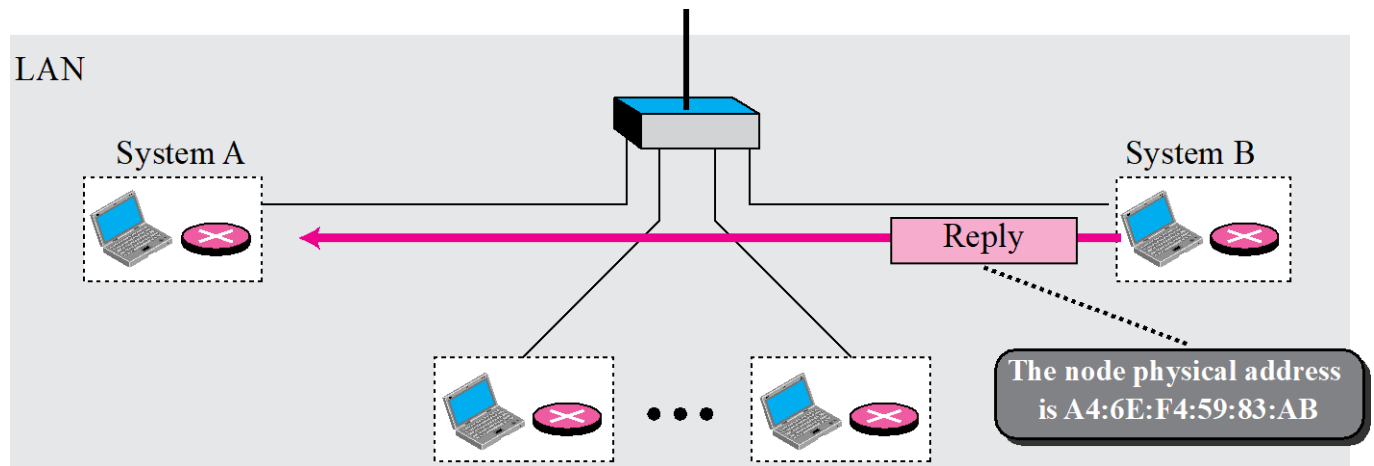


Figure 8.2 *ARP operation*



a. ARP request is broadcast



b. ARP reply is unicast

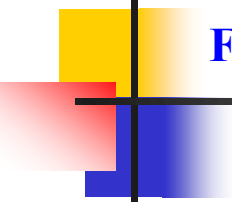
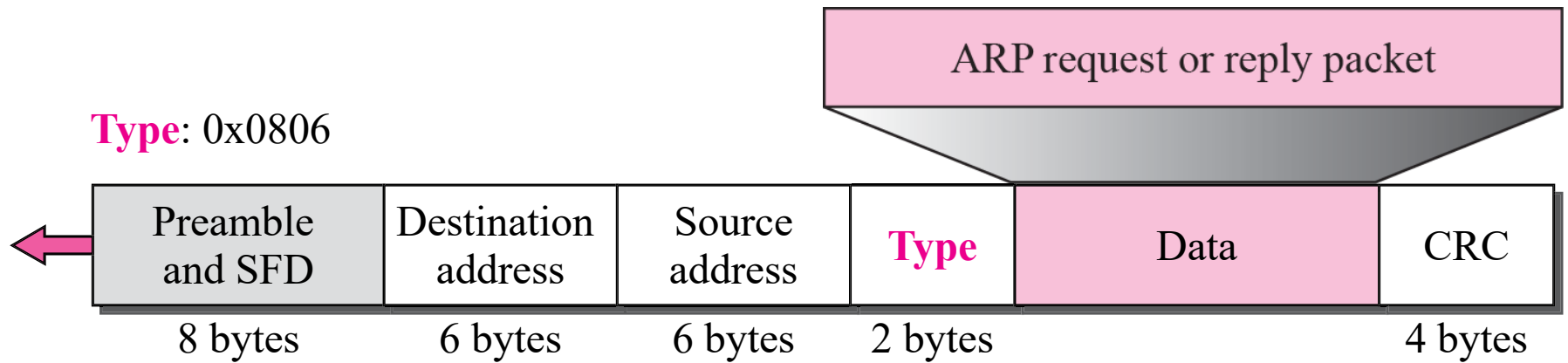


Figure 8.3 *ARP packet*

Hardware Type		Protocol Type
Hardware length	Protocol length	Operation Request 1, Reply 2
Sender hardware address (For example, 6 bytes for Ethernet)		
Sender protocol address (For example, 4 bytes for IP)		
Target hardware address (For example, 6 bytes for Ethernet) (It is not filled in a request)		
Target protocol address (For example, 4 bytes for IP)		

Figure 8.4 *Encapsulation of ARP packet*



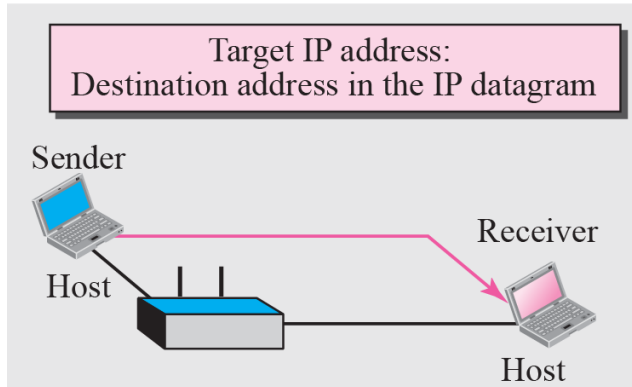


Note

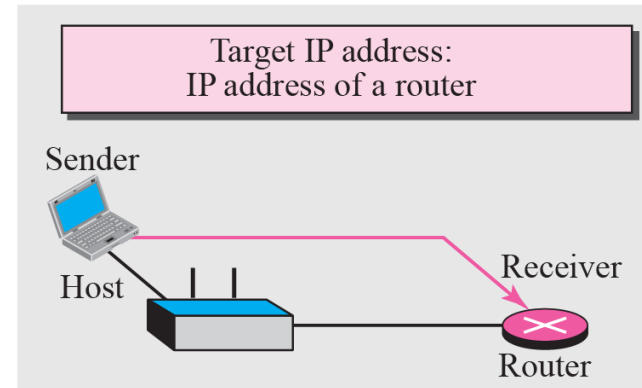
***An ARP request is broadcast;
an ARP reply is unicast.***

Figure 8.5 *Four cases using ARP*

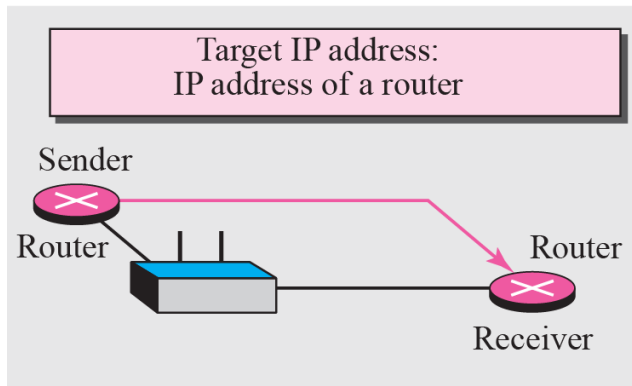
Case 1: A host has a packet to send to a host on the same network.



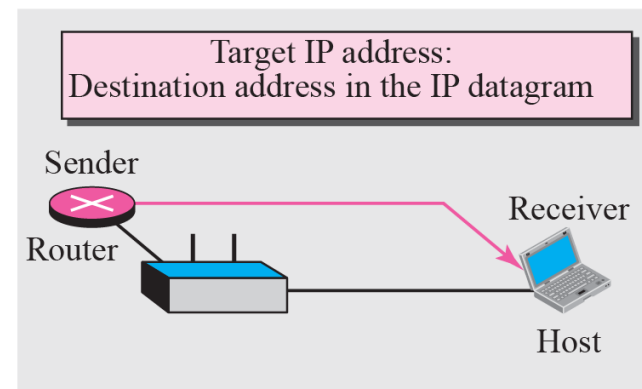
Case 2: A host has a packet to send to a host on another network.



Case 3: A router has a packet to send to a host on another network.



Case 4: A router has a packet to send to a host on the same network.



Example 8.1

A host with IP address 130.23.43.20 and physical address B2:34:55:10:22:10 has a packet to send to another host with IP address 130.23.43.25 and physical address A4:6E:F4:59:83:AB. The two hosts are on the same Ethernet network. Show the ARP request and reply packets encapsulated in Ethernet frames.

Solution

Figure 8.6 shows the ARP request and reply packets. Note that the ARP data field in this case is 28 bytes, and that the individual addresses do not fit in the 4-byte boundary. That is why we do not show the regular 4-byte boundaries for these addresses. Also note that the IP addresses are shown in hexadecimal.

Figure 8.6 Example 8.1

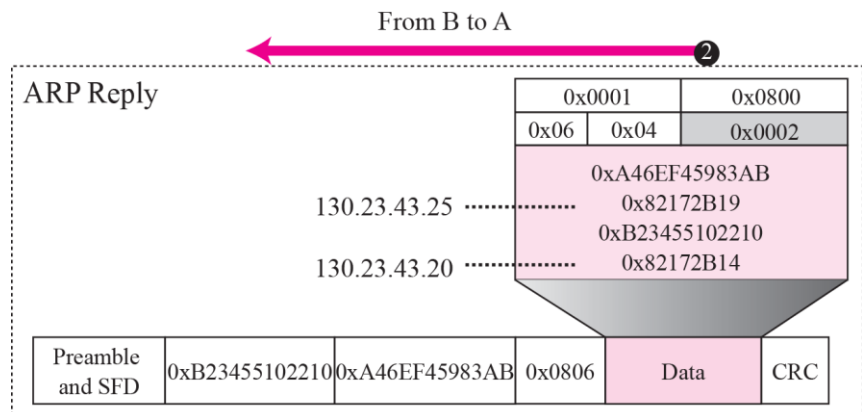
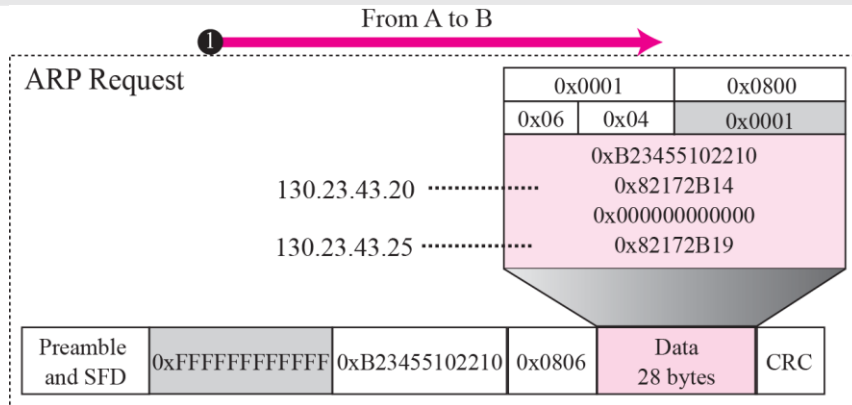
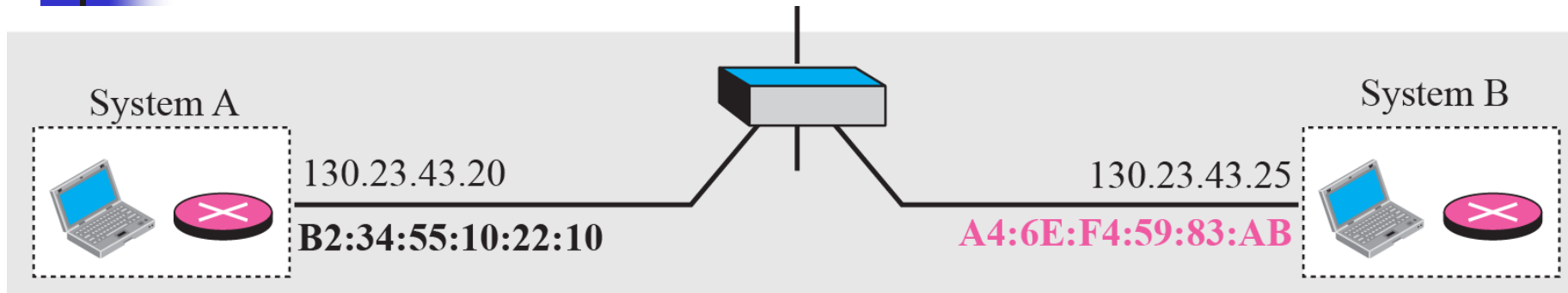
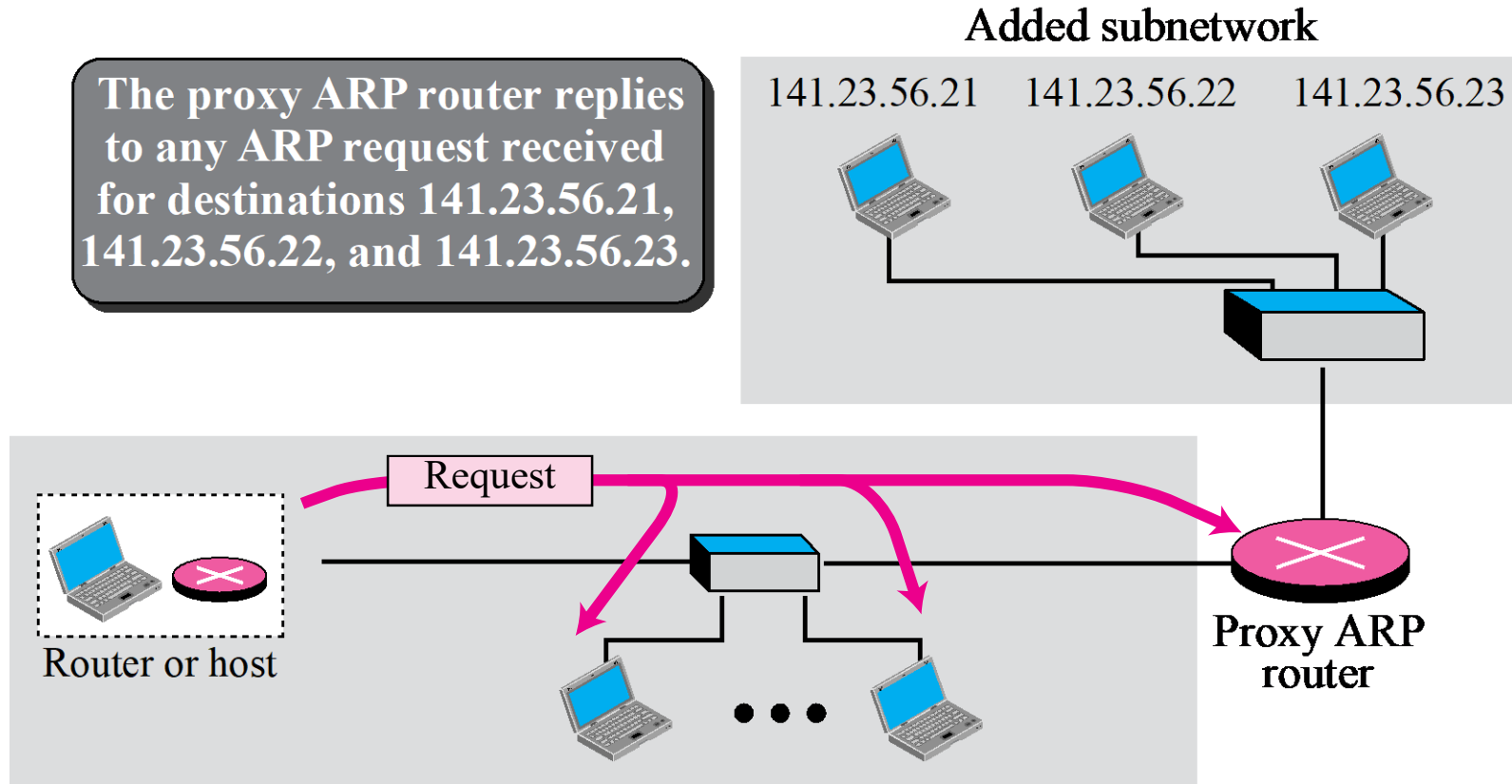


Figure 8.7 *Proxy ARP*



8-3 ATM ARP

We discussed IP over ATM in Chapter 7. When IP packet are moving through an ATMWAN, a mechanism protocol is needed to find (map) the physical address of the exiting-point router in the ATM WAN given the IP address of the router. This is the same task performed by ARP on a LAN. However, there is a difference between a LAN and an ATM network. A LAN is a broadcast network (at the data link layer); ARP uses the broadcasting capability of a LAN to send (broadcast) an ARP request.

Topics Discussed in the Section

- ✓ **Packet Format**
- ✓ **ATMARP Operation**
- ✓ **Logical IP Subnet (LIS)**

Figure 8.8 *ATMARP packet*

Hardware Type		Protocol Type	
Sender Hardware Length	Reserved	Operation	
Sender Protocol Length	Target Hardware Length	Reserved	Target Protocol Length
Sender hardware address (20 bytes)			
Sender protocol address			
Target hardware address (20 bytes)			
Target protocol address			

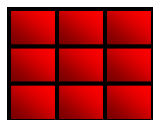


Table 8.1 *OPER field*

<i>Message</i>	<i>OPER value</i>
Request	1
Reply	2
Inverse Request	8
Inverse Reply	9
NACK	10

Note

The inverse request and inverse reply messages can bind the physical address to an IP address in a PVC situation.

Figure 8.9 *Binding with PVC*

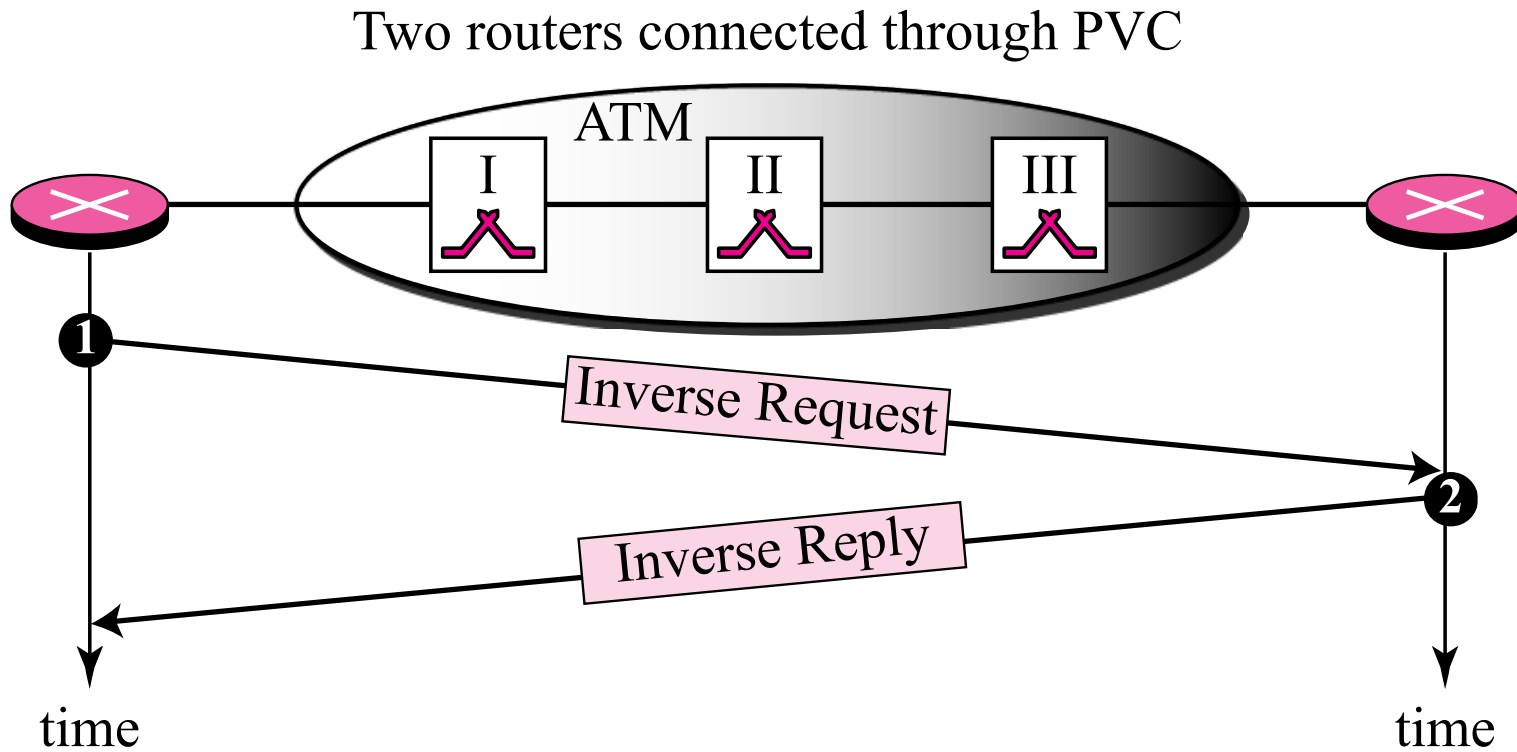
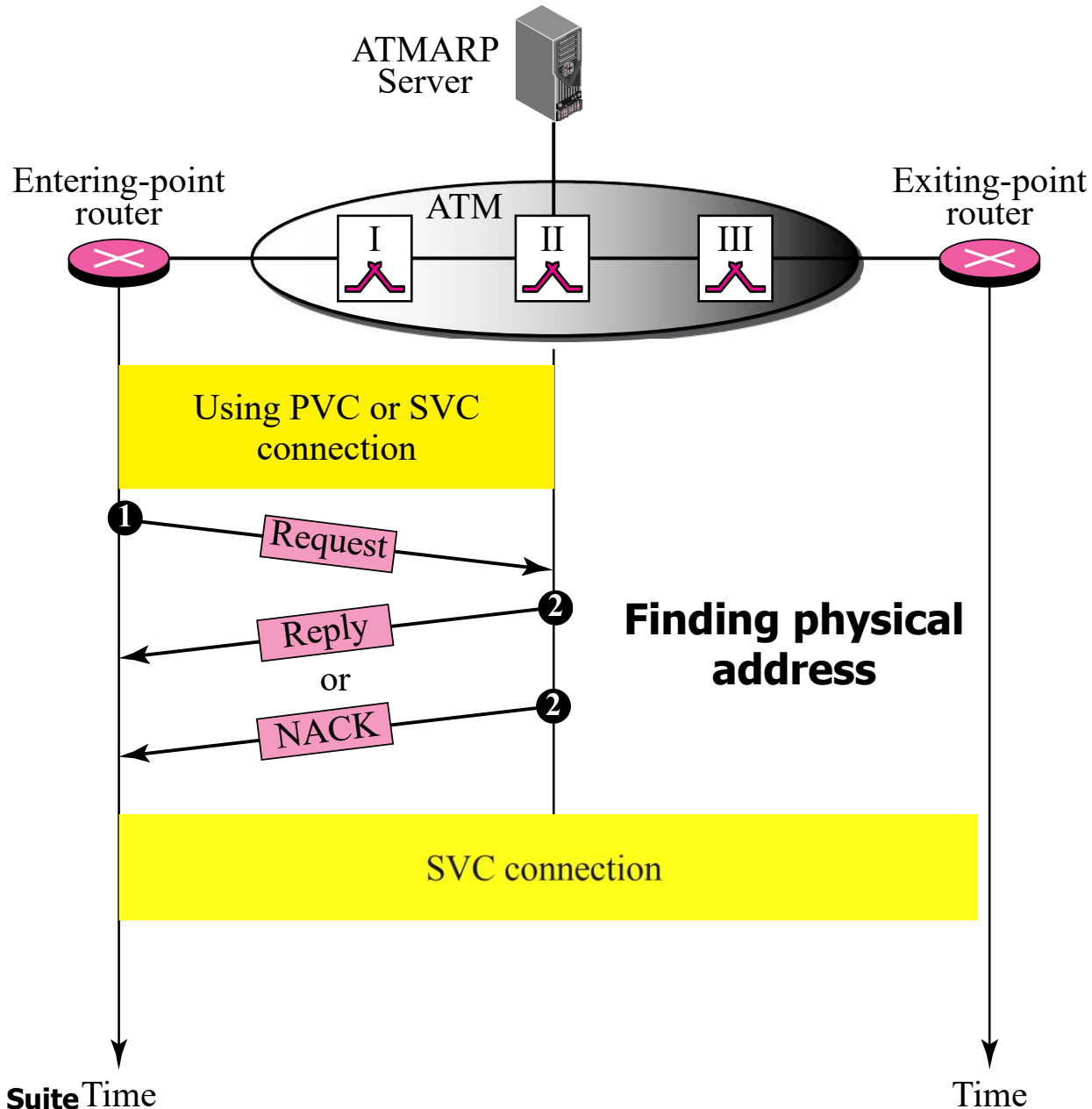


Figure 8.10 *Binding with ATMARP*



Note

The request and reply message can be used to bind a physical address to an IP address in an SVC situation.

Note

***The inverse request and inverse reply
can also be used to build the
server's mapping table.***

Figure 8.11 *Building a table*

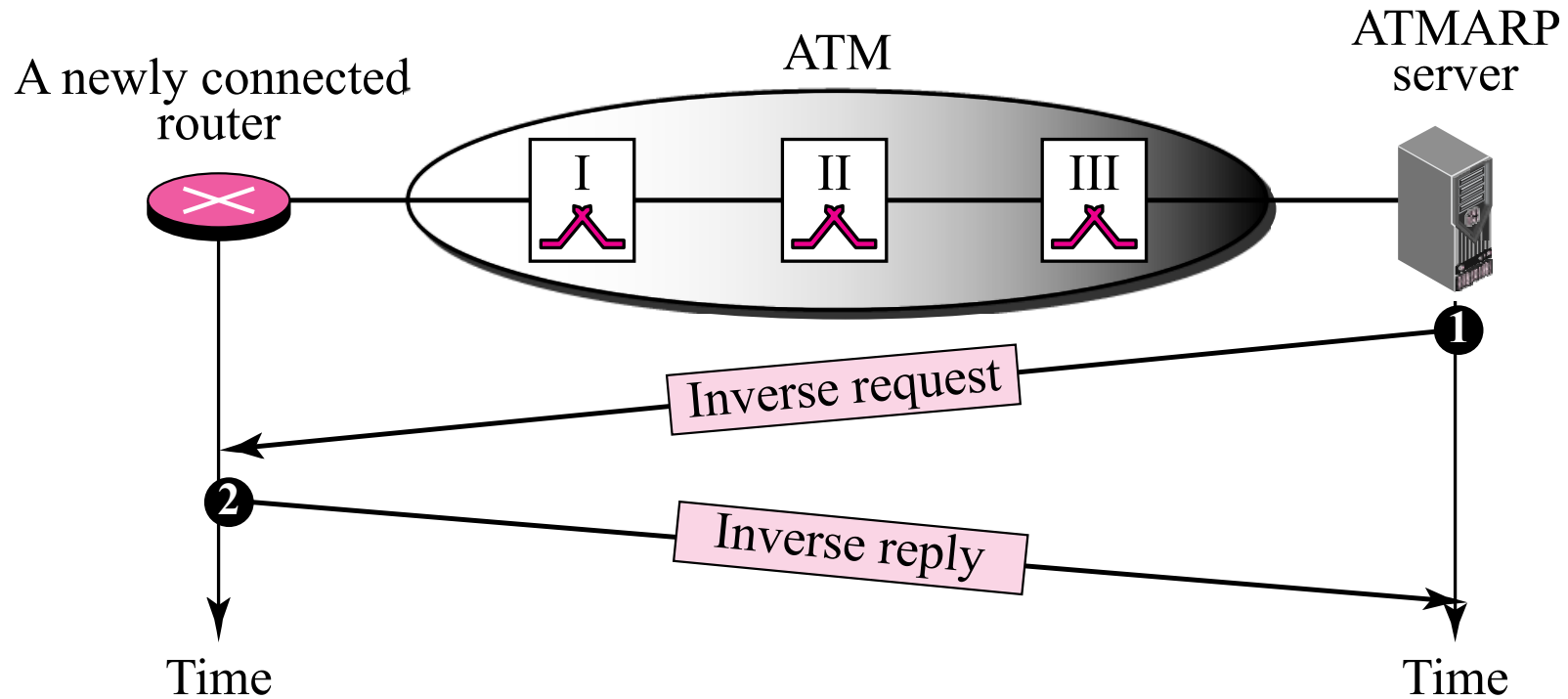
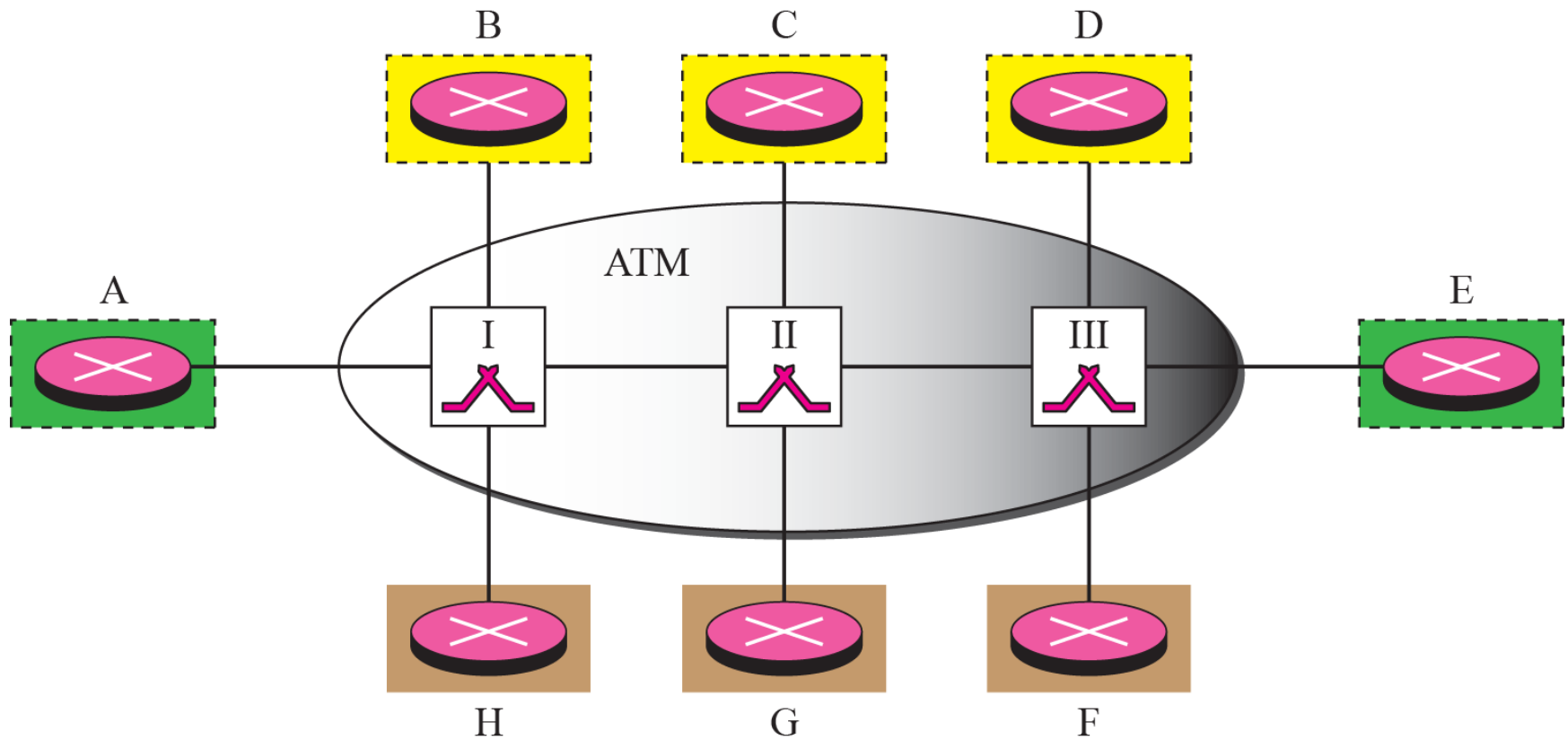


Figure 8.12 *LIS*



Note

LIS allows an ATM network to be divided into several logical subnets.

To use ATMARP, we need a separate server for each subnet.

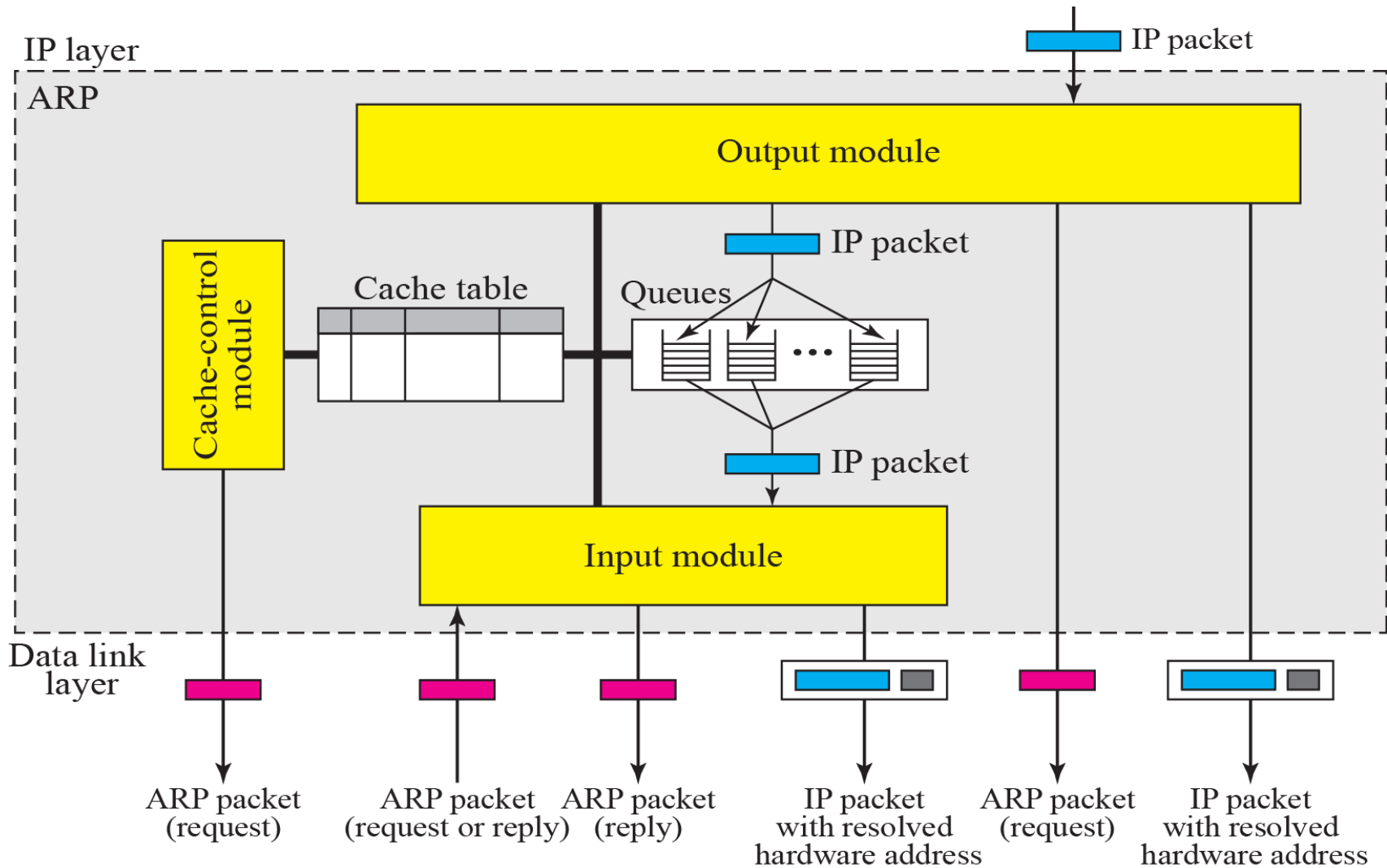
8-4 ARP PACKAGE

In this section, we give an example of a simplified ARP software package. The purpose is to show the components of a hypothetical ARP package and the relationships between the components. Figure 8.13 shows these components and their interactions. We can say that this ARP package involves five components: a cache table, queues, an output module, an input module, and a cache-control module.

Topics Discussed in the Section

- ✓ **Cache Table**
- ✓ **Queues**
- ✓ **Output Module**
- ✓ **Input Module**
- ✓ **Cache-Control Module**

Figure 8.13 *ARP components*



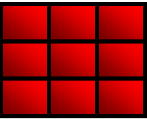


Table 8.2 *Output Module*

```
1  ARP_Output_Module ( )
2  {
3      Sleep until an IP packet is received from IP software.
4      Check cache table for an entry corresponding to the
5          destination of IP packet.
6      If (entry is found)
7      {
8          If (the state is RESOLVED)
9          {
10             Extract the value of the hardware address from the entry.
11             Send the packet and the hardware address to data
12                 link layer.
13             Return
14         } // end if
15         If (the state is PENDING)
16         {
17             Enqueue the packet to the corresponding queue.
18             Return
19         } //end if
20     } //end if
21     If (entry is not found)
22     {
23         Create a cache entry with state set to PENDING and
24             ATTEMPTS set to 1.
25         Create a queue.
26         Enqueue the packet.
27         Send an ARP request.
28         Return
29     } //end if
30 } //end module
```

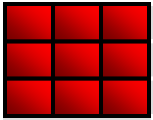



Table 8.3 *Input Module*

```
1  ARP_Input_Module ( )
2  {
3      Sleep until an ARP packet (request or reply) arrives.
4      Check the cache table to find the corresponding entry.
5      If (found)
6      {
7          Update the entry.
8          If (the state is PENDING)
9          {
10             While (the queue is not empty)
11             {
12                 Dequeue one packet.
13                 Send the packet and the hardware address.
14             }//end if
15         }//end if
16     }//end if
17     If (not found)
18     {
19         Create an entry.
20         Add the entry to the table.
21     }//end if
22     If (the packet is a request)
23     {
24         Send an ARP reply.
25     }//end if
26     Return
27 }//end module
```

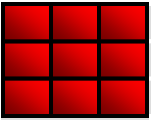


Table 8.4 *Cache-Control Module*

```
1  ARP_Cache_Control_Module ( )
2  {
3      Sleep until the periodic timer matures.
4      Repeat for every entry in the cache table
5      {
6          If (the state is FREE)
7          {
8              Continue.
9          } //end if
10         If (the state is PENDING)
11         {
```

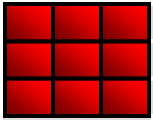


Table 8.4 *Cache-Control Module (continued)*

```
12      Increment the value of attempts by 1.
13      If (attempts greater than maximum)
14      {
15          Change the state to FREE.
16          Destroy the corresponding queue.
17      }// end if
18      else
19      {
20          Send an ARP request.
21      }//end else
22      continue.
23  }//end if
24  If (the state is RESOLVED)
25  {
26      Decrement the value of time-out.
27      If (time-out less than or equal 0)
28      {
29          Change the state to FREE.
30          Destroy the corresponding queue.
31      }//end if
32  }//end if
33  }//end repeat
34  Return.
35  }//end module
```

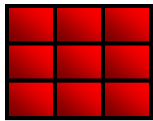


Table 8.5 *Original cache table used for examples*

<i>State</i>	<i>Queue</i>	<i>Attempt</i>	<i>Time-Out</i>	<i>Protocol Addr.</i>	<i>Hardware Addr.</i>
R	5		900	180.3.6.1	ACAE32457342
P	2	2		129.34.4.8	
P	14	5		201.11.56.7	
R	8		450	114.5.7.89	457342ACAE32
P	12	1		220.55.5.7	
F					
R	9		60	19.1.7.82	4573E3242ACA
P	18	3		188.11.8.71	

Example 8.2

The ARP output module receives an IP datagram (from the IP layer) with the destination address 114.5.7.89. It checks the cache table and finds that an entry exists for this destination with the RESOLVED state (R in the table). It extracts the hardware address, which is 457342ACAE32, and sends the packet and the address to the data link layer for transmission. The cache table remains the same.

Example 8.3

Twenty seconds later, the ARP output module receives an IP datagram (from the IP layer) with the destination address 116.1.7.22. It checks the cache table and does not find this destination in the table. The module adds an entry to the table with the state **PENDING** and the **Attempt** value 1. It creates a new queue for this destination and enqueues the packet. It then sends an ARP request to the data link layer for this destination. The new cache table is shown in Table 8.6.

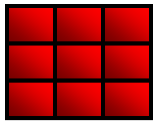


Table 8.6 *Updated cache table for Example 8.3*

<i>State</i>	<i>Queue</i>	<i>Attempt</i>	<i>Time-Out</i>	<i>Protocol Addr.</i>	<i>Hardware Addr.</i>
R	5		900	180.3.6.1	ACAE32457342
P	2	2		129.34.4.8	
P	14	5		201.11.56.7	
R	8		450	114.5.7.89	457342ACAE32
P	12	1		220.55.5.7	
P	23	1		116.1.7.22	
R	9		60	19.1.7.82	4573E3242ACA
P	18	3		188.11.8.71	

Example 8.4

Fifteen seconds later, the ARP input module receives an ARP packet with target protocol (IP) address 188.11.8.71. The module checks the table and finds this address. It changes the state of the entry to RESOLVED and sets the time-out value to 900. The module then adds the target hardware address (E34573242ACA) to the entry. Now it accesses queue 18 and sends all the packets in this queue, one by one, to the data link layer. The new cache table is shown in Table 8.7.

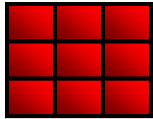


Table 8.7 *Updated cache table for Example 8.4*

<i>State</i>	<i>Queue</i>	<i>Attempt</i>	<i>Time-Out</i>	<i>Protocol Addr.</i>	<i>Hardware Addr.</i>
R	5		900	180.3.6.1	ACAE32457342
P	2	2		129.34.4.8	
P	14	5		201.11.56.7	
R	8		450	114.5.7.89	457342ACAE32
P	12	1		220.55.5.7	
P	23	1		116.1.7.22	
R	9		60	19.1.7.82	4573E3242ACA
R	18		900	188.11.8.71	E34573242ACA

Example 8.5

Twenty-five seconds later, the cache-control module updates every entry. The time-out values for the first three resolved entries are decremented by 60. The time-out value for the last resolved entry is decremented by 25. The state of the next-to-the last entry is changed to FREE because the time-out is zero. For each of the three pending entries, the value of the attempts field is incremented by one. After incrementing, the attempts value for one entry (the one with IP address 201.11.56.7) is more than the maximum; the state is changed to FREE, the queue is deleted, and an ICMP message is sent to the original destination (see Chapter 9). See Table 8.8.

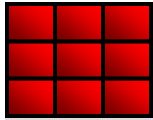


Table 8.8 *Updated cache table for Example 8.5*

<i>State</i>	<i>Queue</i>	<i>Attempt</i>	<i>Time-Out</i>	<i>Protocol Addr.</i>	<i>Hardware Addr.</i>
R	5		840	180.3.6.1	ACAE32457342
P	2	3		129.34.4.8	
F					
R	8		390	114.5.7.89	457342ACAE32
P	12	2		220.55.5.7	
P	23	2		116.1.7.22	
F					
R	18		875	188.11.8.71	E34573242ACA